

Tercera práctica

HUGO TONATIUH CARRILLO BONILLA

320026379



Parte 1: Variables

Actividad 1.1 - Variables en Jshell

Escribiendo las variables una por una.

```
nikte ~
jshell
Welcome to JShell -- Version 21.0.12.1
For an introduction type: /help intro

jshell> int precio = 15000
precio ==> 15000

jshell> double tasa = 12.0
tasa ==> 12.0

jshell> String producto = "Laptop para la carrera"
producto ==> "Laptop para la carrera"

jshell> char tipo = 'P'
tipo ==> 'P'

jshell> boolean disponible = true
disponible ==> true
```

Escribiendo únicamente el nombre de las variables.

```
jshell> precio
precio ==> 15000

jshell> tasa
tasa ==> 12.0

jshell> producto
producto ==> "Laptop para la carrera"

jshell> tipo
tipo ==> 'P'

jshell> disponible
disponible ==> true
```

Cambiando la variable `precio`.

```
jshell> precio
precio ==> 15000

jshell> precio = 18000
precio ==> 18000

jshell> precio
precio ==> 18000

jshell>
```

Al principio, quise ver si **String** se comporta como un dato algebraico que puedes dividir y multiplicar como en lenguajes elegantes y sofisticados. Después ví que se puede operar entre **int** y **float** sin relativamente ningún problema. Se pueden concatenar cadenas con números y además ví que **char** se comporta como un número. Sin embargo, **boolean** no admite valores que no sean estrictamente booleanos.

```
jshell> producto / tasa
Error:
bad operand types for binary operator '/'
first type: java.lang.String
second type: double
producto / tasa
^^^^^^^^^^

jshell> precio / tasa
$14 ==> 1500.0

jshell> producto * 2
Error:
bad operand types for binary operator '*'
first type: java.lang.String
second type: int
producto * 2
^^^^^^^^^^

jshell> producto * (precio / tasa)
$15 ==> "Laptop para la carrera1500.0"

jshell> producto + "" + (precio / tasa)
$16 ==> "Laptop para la carrera1500.0"

jshell> producto + " " + (precio / tasa)
$17 ==> "Laptop para la carrera 1500.0"
```

```
nikte ~
└─ jshell
   Welcome to JShell -- Version 21.0.12.1
   For an introduction type: /help intro

jshell> int a = 'u'
a ==> 117

jshell> int a = true
Error:
Incompatible types: boolean cannot be converted to int
int a = true;
      ^
jshell> boolean b = 1
Error:
Incompatible types: int cannot be converted to boolean
boolean b = 1;
           ^
jshell> byte c = 500
Error:
Incompatible types: possible lossy conversion from int to byte
byte c = 500;
      ^
```

Actividad 1.2 - Agregando variables en P2

Tuve que crear el archivo otra vez porque lo había borrado.

```
nikte ~/practicas P main ✓
└─ ls
   Practica01/ Practica02/ Practica03/ Practica04/ README.md

nikte ~/practicas P main ✓
└─ touch Practica04/Bienvenida.java

nikte ~/practicas P main ?(1) ✓
└─
```

Lo único elegante aquí fue lo que hice con `char`. Para los demás usé `String`. En el nombre la decisión me parece obvia. No así con el número de cuenta. En primera, no sé qué tipo de números de cuenta sean válidos, y por lo tanto puede ser que `int` no alcance. Además, sería peligroso dejar que el usuario manipule **numéricamente** esa información.

```
1 public class Bienvenida {
2     public static void main(String[] args) {
3         String nombre = "Nikte Carrillo Bonilla";
4         String noCuenta = "320026379"; // Esto en realidad es un mal diseño de sistemas: porque el número de cuenta se usa algebráicamente.
5         String expertiz = "Nada";
6         int edad = 22;
7         char inicial = nombre.charAt(0);
8         System.out.println(nombre + " " + noCuenta + " " + expertiz + " " + edad + " " + inicial);
9     }
10 }
```

El programa compiló sin problemas.

```
nikte ~/practicas/Practica4 P main ?(2) ✓
└─ javac Bienvenida.java

nikte ~/practicas/Practica4 P main ?(2) ✓
└─ java Bienvenida
   Nikte Carrillo Bonilla 320026379 Nada 22 N

nikte ~/practicas/Practica4 P main ?(2) ✓
└─
```

Parte 2: Comentarios y expresiones elegantes

Actividad 2.1 - Comentarios en Java

También tuve que volver a descargar el programa `Programa.java`.

```
nikte ~/practicas/Practica4 P main ?(1)?(1) v21.0.12 11:10am
└─ mv /home/nikte/Downloads/Programa.java .

nikte ~/practicas/Practica4 P main ?(1)?(1) v21.0.12 11:14am
└─ ls
Bienvenida.class Bienvenida.java Programa.java

nikte ~/practicas/Practica4 P main ?(1)?(1) v21.0.12 11:14am
└─ javac Programa.java

nikte ~/practicas/Practica4 P main ?(2)?(1) v21.0.12 11:16am
└─ java Programa
=== Ficha de compra ===
- Producto : Laptop para la carrera
- Precio con descuento : 12000
- Plazo de pago en años : 1.5
- Pago mensual : 666.6666666666666
=== Fin de la ficha ===
```

Los comentarios que hice me parecen prudentes porque muchos de mis compañeros tienen mucha curiosidad de qué es lo que pasa realmente con la firma del método principal.

```
1 // Todo programa en Java debe estar dentro de una clase, esta clase debe ser pública
2 public class Programa{
3     // Inicializamos un método en nuestra clase pública, para la JVM
4     // Los métodos estáticos pueden ser invocados sin tener que crear una instancia de la clase
5     // void nos dice que regresa ningún valor
6     // main nos dice que es el método principal que la JVM va a buscar para inicializar
7     // Los argumentos son guardados en un array de Strings
8     public static void main(String[] args) {
9
10        // Inicializamos cuatro variables
11        String producto = "Laptop para la carrera";
12        int precio = 15000;
13        int descuento = 3000;
14        double meses = 18.0;
15
16        // Imprimimos al sistema
17        System.out.println("=== Ficha de compra ===");
18        System.out.println("- Producto : " + producto);
19        System.out.println("- Precio con descuento : " + (precio - descuento));
20        System.out.println("- Plazo de pago en años : " + (meses / 12.0));
21        System.out.println("- Pago mensual : " + ((precio - descuento) / meses));
22        System.out.println("=== Fin de la ficha ===");
23
24    }
25 }
```

Después de hacer comentarios, el programa sigue corriendo bien.

```
nikte ~/practicas/Practica4/Java P main ?(1) v21.0.12 07:50pm
└─ javac Programa.java

nikte ~/practicas/Practica4/Java P main ?(1) v21.0.12 07:51pm
└─ java Programa
=== Ficha de compra ===
- Producto : Laptop para la carrera
- Precio con descuento : 12000
- Plazo de pago en años : 1.5
- Pago mensual : 666.6666666666666
=== Fin de la ficha ===

nikte ~/practicas/Practica4/Java P main ?(1) v21.0.12 07:51pm
└─
```

Actividad 2.2 - Printf

Consulté una página de internet para resolver esta actividad [1]. `printf` utiliza placeholders precedidos con `%` para distintos tipos de datos. Para `float`, podemos controlar el número de dígitos después del punto decimal con `%.2f`.

Renombré el archivo sin saber que también tenía que subir el archivo con comentarios.

Referencias

- [1] W3 Schools. *Java Output printf() Method*. URL: https://www.w3schools.com/JAVA/ref_output_printf.asp. (accesado: 29.09.2026).

Parte 3: Reto de abstracción

Actividad 3.1 - Filtrando información

He aquí una lista condensada de los datos que aprecié como necesarios y como innecesarios. **Datos necesarios:**

- Precio de la laptop.
- En concreto, el precio de la laptop de nuestro primer cliente
- La cantidad de mensualidades en las que van a pagar
- Interés anual del 15 % .
- Pago en total.

El precio es indispensable para saber cuánto cobraremos, en especial el de Robbie, por esta misma razón debemos de saber en cuántas mensualidades nos va a pagar y su interés. Técnicamente el pago total lo sabremos nosotros, y no es un dato que se nos dé, sino que lo tenemos que deducir con la información dada.

Datos innecesarios:

- Especificaciones de la laptop.
- La calificación del cliente.

Las especificaciones de la laptop nos son indiferentes, el cliente — Robbie — nos va a pagar incluso si su laptop la comprara descompuesta. Y la calificación del cliente, aunque es importante, no es vital pues no sabemos todavía la forma en la que su calificación afectará sus mensualidades o su interés.

De lejos, el dato que más me costó clasificar fue precisamente el de la calificación del cliente, porque a la vez tanto es importante para el subsecuente desarrollo de nuestro programa, como nos es superfluo para el problema a mano. Implementarlo además excede nuestras capacidades técnicas — requeriríamos hacer un objeto `cliente` con la calificación como atributo — y eso complicaría innecesariamente nuestra base de código demasiado pronto en el ciclo de desarrollo.

Actividad 3.2 - Cumpliendo los deseos del señor Pines

Mi implementación me parece sumamente mediocre y subpar. La falta de herramientas de control de flujo lleva inevitablemente a que calculemos nuestro interés en una sola entrega en lugar de calcularlo año por año.



```
nikte ~/practicas/Practica4 P cotizador ++(2) v21.0.12 06:42pm
└─ javac Cotizador.java

nikte ~/practicas/Practica4 P cotizador ++(2) v21.0.12 06:42pm
└─ java Cotizador
Mensualidades: 614.00
Interés: 1934.85
Pago total: 14833.85

nikte ~/practicas/Practica4 P cotizador ++(2) v21.0.12 06:42pm
└─ |
```

Conclusión

Esta práctica es muy interesante porque por fin empezamos a resolver problemas de programación y es una gran motivación para empezar a aprender métodos tradicionales y procedurales de programación, como también nos deja atisbar la clase (ja, ja) de problemas que podremos resolver con objetos. Finalmente, pero no por ello menos importante, aprendí que debo empezar a leer con cuidado las instrucciones en general y los *verbos* que se utilizan en particular, pues más de una vez tuve que realizar una tarea en múltiples ocasiones porque borraba, editaba y renombraba archivos que debía de dejar intactos.